Aufgabe: Polymorphie (Polymorphism)

Für: Michael **Referenz:** W3Schools – C# Polymorphism **Bearbeitungszeit:** ca. 45–60 Min. **Voraussetzung:** Diese Aufgabe baut direkt auf deiner **Vererbungsaufgabe** (`Fahrzeug` , `Auto` , `Motorrad`) auf. Neu ist nur **Polymorphie**.

---

1. Was du schon kannst (kurzer Rückblick)

- **Vererbung:** `Auto : Fahrzeug` – eine abgeleitete Klasse erbt Felder/Methoden der Basisklasse
- **Konstruktor +** `base(...)` **:** reicht Werte an den Konstruktor der Basisklasse weiter
- **Access Modifiers & Properties:** `private` -Feld, kontrollierter Zugriff über `public` -Property
- **Arrays/Listen &** `foreach` **:** eine Sammlung von Elementen durchlaufen, ohne jedes einzeln anzusprechen

Genau diese vier Bausteine kombinierst du jetzt mit dem neuen Konzept.

2. Neu: Polymorphie

Lies zuerst das W3Schools-Kapitel: **C# Polymorphism**.

Die Kernidee laut W3Schools: *“Polymorphism means ‘many forms’, and it occurs when we have many classes that are related to each other by inheritance.”* Vererbung stellt die Beziehung her – Polymorphie nutzt sie, um **eine Handlung auf viele Arten** auszuführen.

Das Überraschungs-Beispiel

W3Schools zeigt zuerst absichtlich ein Beispiel, das **nicht** wie erwartet funktioniert:

```
class Animal
{
  public void animalSound()
  {
    Console.WriteLine("The animal makes a sound");
  }
}

class Pig : Animal
{
  public void animalSound()
  {
    Console.WriteLine("The pig says: wee wee");
  }
}
```

```
Animal myPig = new Pig();
myPig.animalSound();
```

Vorhersage zuerst: Was, glaubst du, gibt das aus? Schreib deine Vermutung auf. (Spoiler: W3Schools nennt das Ergebnis selbst *“Not The Output I Was Looking For”* – es kommt **“The animal makes a sound”**, nicht der Schweine-Laut!)

Die Lösung: `virtual` und `override`

```
class Animal
{
```

```
  public virtual void animalSound()   // virtual erlaubt das Überschreiben
  {
    Console.WriteLine("The animal makes a sound");
  }
}

class Pig : Animal
{
  public override void animalSound()  // override ersetzt es tatsächlich
  {
    Console.WriteLine("The pig says: wee wee");
  }
}
```

Jetzt gibt `myPig.animalSound()` korrekt **“The pig says: wee wee”** aus.

Merksatz: Ohne `virtual` / `override` “gewinnt” die Basisklasse immer. Erst `virtual` (Basisklasse: “darf überschrieben werden”) + `override` (Kindklasse: “wird jetzt ersetzt”) macht Polymorphie möglich.

---

3. Deine Aufgabe: Polymorphie in der Fahrzeug-Hierarchie

Du erweiterst `Fahrzeug`, `Auto` und `Motorrad` aus der letzten Aufgabe.

Schritt 1 – Das Überraschungs-Beispiel nachbauen

Füge `Fahrzeug` eine ganz normale (noch **nicht** `virtual`) Methode hinzu:

```
public void Hupen()
{
    Console.WriteLine("Tuut, tuut!");
}
```

Überschreibe sie testweise gleich benannt in `Auto` – **ohne** `virtual` / `override` (das gibt ohnehin einen Compiler-Hinweis, teste es trotzdem einmal bewusst mit `new` statt `override`, falls der Compiler das verlangt). Rufe sie so auf:

```
Fahrzeug meins = new Auto("Ford", "Mustang");
meins.Hupen();
```

Vorhersage zuerst: Was gibt das aus – der Fahrzeug-Text oder der Auto-Text? Notiere deine Vermutung, dann ausführen.

Schritt 2 – Mit `virtual` / `override` reparieren

Ändere `Hupen()` in `Fahrzeug` zu `virtual`, und in `Auto` sowie `Motorrad` zu `override`. Gib jeder Klasse einen eigenen, sinnvollen Text (z.B. Auto: `"Hup, hup!"`, Motorrad: `"Brumm!"`).

Führe denselben Aufruf wie in Schritt 1 erneut aus.

Leitfrage: Was hat sich am Aufruf-Code (`meins.Hupen();`) geändert? Was hat sich an der Ausgabe geändert? Warum?

Schritt 3 – Die eigentliche Stärke: eine gemischte Liste

```
List<Fahrzeug> meineFahrzeuge = new List<Fahrzeug>();
meineFahrzeuge.Add(new Auto("Ford", "Mustang"));
meineFahrzeuge.Add(new Motorrad("Honda", 650));

foreach (Fahrzeug f in meineFahrzeuge)
{
    f.Hupen();
```

```
        }
```

Vorhersage zuerst: Wie viele Zeilen gibt das aus? Was steht in welcher Zeile?

Leitfrage (die wichtigste dieser Aufgabe): Die Schleife kennt nur den Typ `Fahrzeug`. Woher "weiss" jedes Objekt trotzdem, welchen eigenen Text es ausgeben soll?

Schritt 4 – Erweitern, ohne die Schleife anzufassen

Baue eine dritte Klasse `Fahrrad : Fahrzeug` mit eigenem `Hupen()` (z.B. `"Kling, kling!"`). Füge ein `Fahrrad`-Objekt der Liste aus Schritt 3 hinzu.

Regel: Du darfst dafür **keine einzige Zeile** in der `foreach`-Schleife ändern.

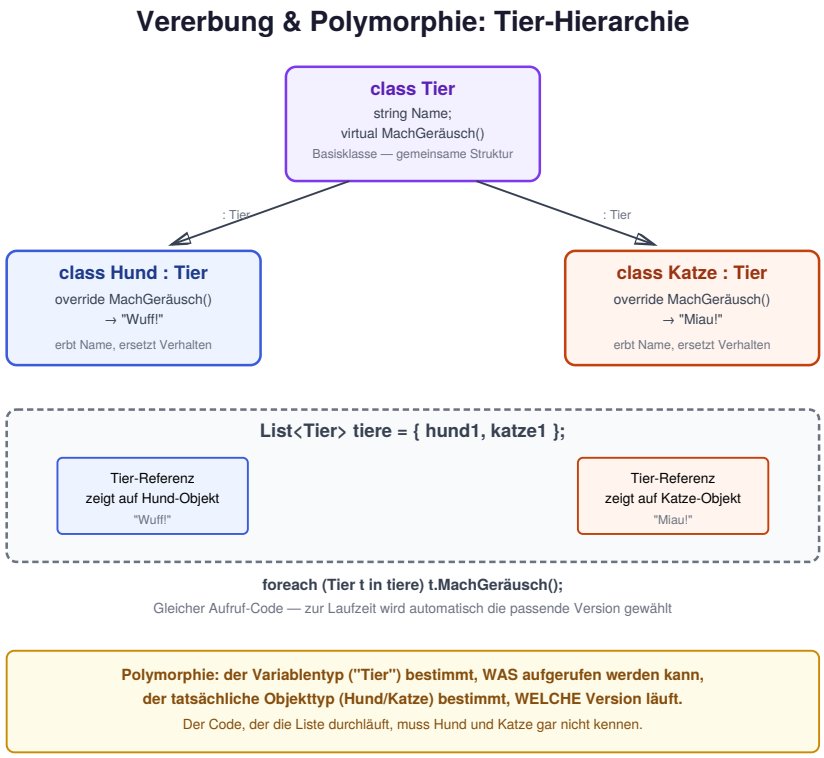
Leitfrage: Warum funktioniert das, ohne dass du den Schleifen-Code anfassen musstest?

---

4. Stolperstein-Check

- ☐ Hast du `virtual` in `Fahrzeug` **und** `override` in **jeder** Kindklasse gesetzt? (Fehlt eins von beiden, gibt's einen Compilerfehler oder das alte, falsche Verhalten.)
- ☐ Hast du in Schritt 1 wirklich den Unterschied *ohne* `virtual` / `override` gesehen, bevor du repariert hast? (Der Kontrast ist der Lerneffekt.)
- ☐ Kannst du erklären, warum `List<Fahrzeug>` sowohl `Auto`- als auch `Motorrad`- und `Fahrrad`-Objekte enthalten darf?

5. Bild zur Orientierung



Vererbung und Polymorphie

*Diesmal zählt vor allem der **untere** Teil des Bildes: eine `List<Tier>` mit gemischten Objekten, `foreach` ruft bei jedem automatisch die passende, überschriebene Version auf – genau das hast du gerade mit deinen Fahrzeugen gebaut.*

Guter Alltagsvergleich: Ein Lichtschalter ist immer derselbe Knopf – je nach Raum schaltet er aber eine andere Lampe. Der Schalter selbst muss die Lampe nicht kennen.

---

6. Abschluss-Fragen (mündlich beantworten)

1. Was passiert, wenn `Hupen()` in `Fahrzeug` **kein** `virtual` hat, aber `Auto` es trotzdem mit demselben Namen neu schreibt?
2. Was bewirkt `virtual` in der Basisklasse, was bewirkt `override` in der Kindklasse?
3. Warum darf `meineFahrzeuge` (Typ `List<Fahrzeug>`) auch `Motorrad`- und `Fahrrad`-Objekte enthalten?
4. Was müsstest du am `foreach`-Code ändern, wenn du eine vierte Fahrzeugart hinzufügst? (Antwort: nichts – das ist der Punkt.)

7. Weiterführend

Wenn das sitzt, geht es weiter mit **C# Abstraction** – dort lernst du, wie man erzwingt, dass jede Kindklasse eine Methode wie `Hupen()` überhaupt überschreiben **muss**, statt es nur zu dürfen.